

FDE研修 新卒コース ハンズオン教材(全9章)

画面イメージと作業手順で学ぶ、12週間の実践テキスト

対象:新卒コース受講者(プログラミング基礎習得済み・実務経験なし)/教材リポジトリ

リ:company_inner_search_app2

関連資料:docs/fde-training-program.md(カリキュラム全体)、docs/fde-training-beginner-handbook.html(未経験者コース教材)

本書の使い方

- 本書は新卒コース(12週間)を **9つの章**に分け、「学習ゴール → 作業手順 → 画面イメージ → 確認ポイント」の順で解説します。Python・Gitの文法解説は含みません(不安がある場合は未経験者コース教材の第1〜2章で補習)。
- このコースのテーマは「動くコードが書ける」から「実務で通用する」への進化です。個々の課題は「提出して終わり」ではなく、レビュー・発表・振り返りまでがセットです。
- 画面イメージ(▲マークの図)は、その手順を正しく実行できたときに見えるはずの画面です。

第1週

①チーム開発

第2週

②コード読解

第3週

③テスト・ログ

第4週

④RAG評価

第5週

⑤オントロジー

第6週

⑥パイプライン

第7週

⑦デプロイ

第8〜9週

⑧コンサル実践

第10〜12週

⑨模擬プロジェクト

目次

- 第1章 チーム開発の作法(第1週)**
Issue駆動開発/ブランチ・コミット・PRの規約
- 第2章 コードリーディング(第2週)**
教材アプリの責務分離を読み解き、図解する
- 第3章 テスト・ログ・エラーハンドリング(第3週)**
pytestでutils.pyにテストを追加する
- 第4章 RAGの内部理解とチューニング評価(第4週)**
チャンクサイズ・top-k・モデルの比較実験
- 第5章 データ基盤とオントロジー(第5週)**
Palantir OSDK検証スクリプトを題材に学ぶ
- 第6章 データパイプライン演習(第6週)**
雑多なファイル群を構造化データに変換する

7. **第7章** クラウドデプロイ基礎(第7週)

Streamlit Cloudへの公開とシークレット管理

8. **第8章** コンサルティング実践(第8～9週)

要件が変わるロールプレイ/シャドーイング

9. **第9章** 模擬プロジェクト(第10～12週)

チーム開発・週次デモ・360度フィードバック

第1章 チーム開発の作法

個人開発とチーム開発の最大の違いは「作業がすべて記録され、他人に引き継げる形になっていること」。Issue駆動の開発サイクルを初週で体に入れます。

🎯 この章の学習ゴール

- 作業を始める前に Issue を起票し、「何を・なぜやるか」を文書化できる
- チームの規約(ブランチ名・コミットメッセージ・PR説明)に沿って開発できる
- レビューで指摘する側・される側の両方を経験する

1-1. Issue 駆動開発のサイクル

今週は「教材アプリの改善タスク」を毎日1件、次のサイクルで回します。コードを書く前に必ず Issue を書くのがルールです。

1 Issue を起票する

GitHub リポジトリの「Issues」タブ → 「New issue」。背景(なぜ)・やること(何を)・完了条件(どうなったら終わりか)の3点セットで書きます。

https://github.com/koheiashikari/company_inner_search_app2/issues/new

koheiashikari / company_inner_search_app2

起動時のエラーメッセージに対処方法へのリンクを追加したい

背景(なぜ)

環境構築でエラーになった研修生が、原因の切り分けに平均30分かかっている。

やること(何を)

初期化失敗時のエラーメッセージに、社内Wikiのトラブルシューティングページへの案内文を追加する。

完了条件

- [] `.env` 未設定の状態で起動し、案内文つきエラーが表示される
- [] 正常時の動作に影響がない

Submit new issue

▲ **画面イメージ:** Issueの記入例。完了条件をチェックボックスで書いておくと、PRレビュー時に「できた/できていない」の判断が明確になります。

2 規約に沿ってブランチを切り、コミットする

ブランチ名は `種類/issue番号-内容`、コミットメッセージは `1行目に要約、3行目以降に理由` がチーム規約です。

```
git switch -c fix/23-error-message-guide
# --- 編集後 ---
git add components.py
git commit -m "初期化エラー時にトラブルシューティングページへの案内を追加

環境構築でつまづいた際の自己解決率を上げるため。
関連Issue: #23"
```

3 PR を出し、Issue と紐づける

PR説明の冒頭に `Closes #23` と書くと、マージ時にIssueが自動で閉じます。「変更内容・確認方法・スクリーンショット」を必ず添付します。

相互レビュー:指摘する側も経験する

今週は同期のPRを**毎日1件レビュー**します。観点:①Issueの完了条件を満たすか ②動作確認手順で再現できるか ③命名やコメントは他人が読んで分かるか。指摘は「提案形」で書きます(例:「～にする
とより意図が伝わりそうです」)。

✓ 確認ポイント(第1週修了の条件)

- Issue起票 → PR → マージのサイクルを5回完走した(1日1件)
- 同期のPRに、相手が「なるほど」と返したレビューコメントを5件以上書いた
- 自分のコミット履歴だけを見て、1週間の作業内容を口頭で再現できる

第2章 コードリーディング

実務では「書く時間」より「読む時間」の方が長い。教材アプリの責務分離を読み解き、アーキテクチャ図を作成して発表します。

🎯 この章の学習ゴール

- 初見のコードベースを「入口から」体系的に読む手順を身につける
- 教材アプリの4ファイルの責務と依存関係をA4一枚の図に整理できる
- 「なぜこう分けたのか」という設計意図を推測し、自分の言葉で説明できる

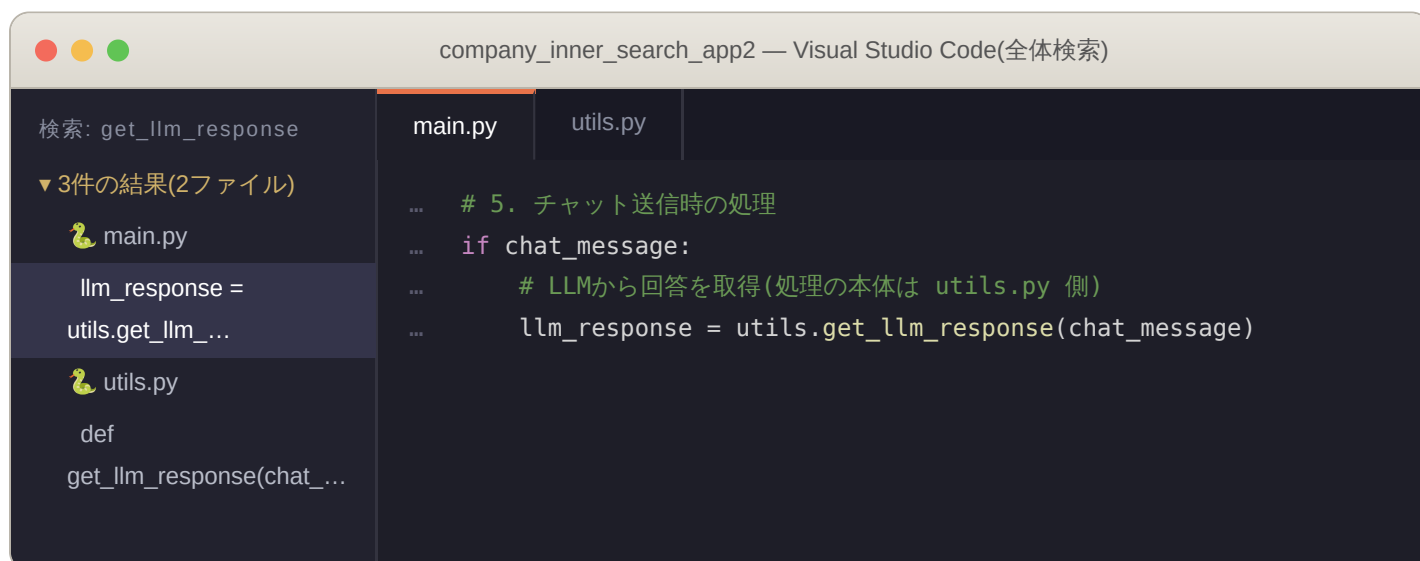
2-1. 読解の手順

1 入口(main.py)から実行順に読む

アプリの入口 `main.py` を上から読み、「起動してから回答が表示されるまで」に呼ばれる関数を紙に書き出します。読みながらVS Codeの「定義へ移動」(F12)で関数の中身へ潜り、「戻る」(Alt+←)で帰ってくるのが基本動作です。

2 「どこで使われているか」を横断検索する

気になる関数や定数(例: `get_llm_response`、`st.session_state`)を全体検索(Ctrl+Shift+F)し、定義と利用箇所の対応表を作ります。

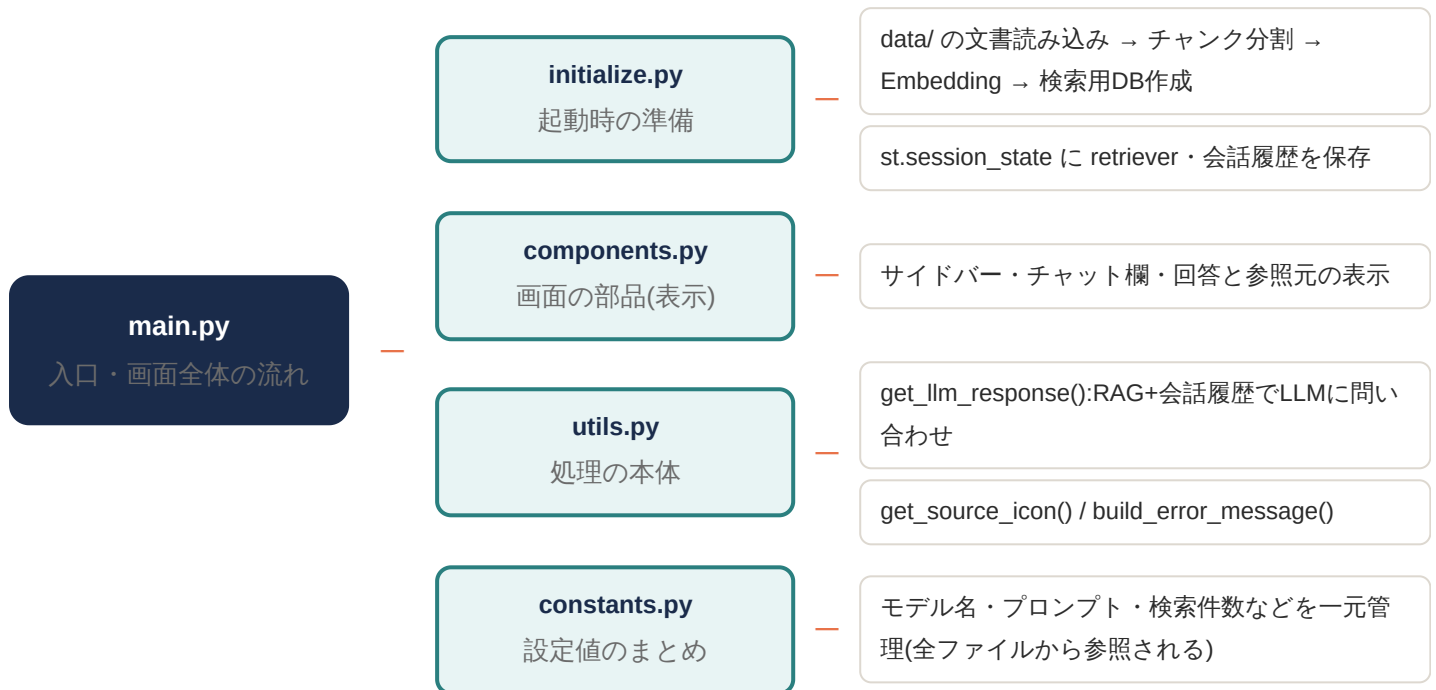


▲ 画面イメージ: 全体検索で「定義1か所・利用1か所」と分かれば、`main.py` は流れの制御だけを担い、処理の本体は `utils.py` にある——という責務分離が見えてきます。

3

アーキテクチャ図を描いて発表する(金曜)

調べた内容をA4一枚の図にまとめ、10分で発表します。箱=ファイル、矢印=呼び出し・参照の向き。「もし〇〇を変更したら、どのファイルに触ることになるか」という質問に図で答えられることが合格ラインです。



▲ **画面イメージ**: アーキテクチャ図の完成イメージ。「設定を変えたいなら constants.py だけ」「見た目なら components.py だけ」——変更が局所化されるのが良い設計、という気づきまで到達するのが理想です。

✓ **確認ポイント(第2週修了の条件)**

- 「質問を1回送信すると、どのファイルのどの関数がどの順で動くか」を図なしで口頭説明できる
- 発表で受けた質問のうち、答えられなかったものを翌日までに調べて回答した

第3章 テスト・ログ・エラーハンドリング

「動く」と「壊れていないと言い切れる」は別物。教材リポジトリの `test_parantia_ontology.py` を参考に、`utils.py` へ `pytest` のテストを追加します。

🎯 この章の学習ゴール

- `pytest` でテストを書き、自動実行できる
- 「テストしやすい関数・しにくい関数」の違い(外部依存の有無)を説明できる
- 障害発生を想定したログ・エラーメッセージの設計観点を持つ

3-1. `pytest` でテストを書く

1 テスト対象を選ぶ:まず「純粋な関数」から

`utils.py` には3つの関数があります。`get_llm_response()` はLLM・画面状態に依存するためテストが難しい。一方 `get_source_icon()` と `build_error_message()` は「入力を渡せば出力が決まる」**純粋な関数**なので、最初のテスト対象に最適です。

2

テストコードを書く

tests/test_utils.py を作成します。テスト関数名は test_〇〇 で始め、「正常系」だけでなく「境界・異常系」も書くのがポイントです。

tests/test_utils.py

```
import utils
import constants as ct

# --- get_source_icon: 参照元の種類でアイコンが切り替わるか ---
def test_get_source_icon_web():
    # Webページ(http始まり)ならリンク用アイコン
    assert utils.get_source_icon("https://example.com/rule") == ct.LINK_SOURCE_ICON

def test_get_source_icon_file():
    # ファイルパスなら文書用アイコン
    assert utils.get_source_icon("data/社員について/社員名簿.csv") == ct.DOC_SOURCE_ICON

def test_get_source_icon_tricky():
    # 境界ケース: 「http」を途中に含むだけのファイル名は文書扱いのはず
    assert utils.get_source_icon("data/httpサーバー設定.txt") == ct.DOC_SOURCE_ICON

# --- build_error_message: 問い合わせ文が連結されるか ---
def test_build_error_message():
    result = utils.build_error_message("初期化に失敗しました")
    assert result.startswith("初期化に失敗しました")
    assert ct.COMMON_ERROR_MESSAGE in result
```

3

実行して全件PASSさせる

```
pip install pytest
pytest tests/ -v
```

```
(.venv) PS ...\.company_inner_search_app2> pytest tests/ -v
===== test session starts =====
collected 4 items

tests/test_utils.py::test_get_source_icon_web PASSED [ 25%]
tests/test_utils.py::test_get_source_icon_file PASSED [ 50%]
tests/test_utils.py::test_get_source_icon_tricky PASSED [ 75%]
tests/test_utils.py::test_build_error_message PASSED [100%]
===== 4 passed in 0.42s =====
(.venv) PS ...\.company_inner_search_app2> _
```

▲ **画面イメージ:** 全件PASSの状態。以後、コードを変更するたびにこのコマンド一発で「壊していないこと」を確認できます。わざと utils.py を壊してFAILの画面も見ておきましょう。

3-2. ログとエラーハンドリングのレビュー演習

1 「深夜に障害対応する自分」を想像してログを読む

教材アプリを起動し、わざと `.env` を外す・data/ を空にする、などの異常を起こしてログ出力を観察します。次のチェックリストで評価し、改善案を1件PRにします。

チェック観点	悪い例 → 良い例
いつ・どこで・何が起きたか分かるか	「エラーが発生しました」→「初期化処理でdata/の読み込みに失敗(対象: 社員名簿.csv)」
利用者向けと開発者向けが分離されているか	画面には対処方法を、ログには技術的詳細(スタックトレース)を出す
秘密情報が漏れていないか	APIキーや個人情報をログに出力しない

✓ 確認ポイント(第3週修了の条件)

- tests/ のテストが4件以上あり、全件PASSしている(PR提出済み)
- 「get_llm_response() のテストはなぜ難しいか」を外部依存という言葉で説明できる
- ログ改善のPRが1件マージされた

第4章 RAGの内部理解とチューニング評価

「なんとなく動く」から「根拠を持って調整できる」へ。チャンクサイズ・検索件数(top-k)・Embeddingモデルを変えて回答品質を比較し、評価レポートを作成します。

🎯 この章の学習ゴール

- RAGの品質を決める3つのパラメータの役割とトレードオフを説明できる
- 評価用の質問セットを設計し、条件を変えて定量比較できる
- 実験結果から「この設定を推奨する。理由は～」と意思決定できる

4-1. 実験の設計

1 評価用の質問セット(10問)を作る

data/ の文書から答えられる質問を10問、**正解(期待する回答)つき**で用意します。簡単な事実確認(「〇〇さんの部署は?」)から、複数文書をまたぐ質問(「営業部で議事録に登場した社員は?」)までレベルを混ぜます。

2 変更するパラメータと実験計画を決める

設定値は `constants.py` に集約されています(第2章の読解が活きる場面)。**1回の実験で変えるのは1つだけ**が実験の鉄則です。

パラメータ	試す値	予想されるトレードオフ
チャンクサイズ	300 / 500 / 1000文字	小さい=ピンポイントだが文脈が切れる/ 大きい=文脈は残るがノイズ混入
検索件数(top-k)	3 / 5 / 10件	少ない=根拠不足のリスク/多い=無関係 文書が回答を乱す
Embeddingモデル	small / large	大きいモデルは精度が上がるが、コスト と処理時間が増える

3 実験 → 採点 → レポート

各条件で10問を実行し、**正確性(0~2点)**と**根拠の適切さ(0~2点)**で採点して記録します。金曜に「推奨設定と、その根拠」を1枚にまとめて発表します。

RAGチューニング評価レポート(記入例・抜粋)

実験者:研修生 | 質問セット:10問 | 評価:正確性0〜2点 + 根拠0〜2点(満点40点)

条件(変更点のみ)	スコア	気づき
基準設定(500字 / k=5 / small)	31 / 40	単純な事実確認は全問正解。議事録をまたぐ質問で根拠漏れ
チャンク300字	27 / 40	表データの行が泣き別れになり、社員名簿系の質問が悪化
チャンク1000字	29 / 40	回答に無関係な文が混ざる。参照元の特定が曖昧に
k=10	28 / 40	「それらしいが違う」文書が根拠に混入。回答が長くなり焦点がぼける
結論	500字 / k=5 を推奨。表形式データが多い本データセットではチャンクを小さくする恩恵がなく、kを増やすとノイズが支配的になるため。	

▲ **画面イメージ:** 評価レポートの完成イメージ。数字と「なぜそうだったか」の考察がセットになっていることが重要。この形式は実案件の精度改善レポートでそのまま使えます。

✔ **確認ポイント(第4週修了の条件)**

- 3パラメータ×計5条件以上の実験を完了し、スコアを記録した
- 「顧客に費用を抑えたいと言われたら、どの設定をどう妥協するか」に即答できる

第5章 データ基盤とオントロジー(Palantir OSDK)

個々のアプリから一段上がり、「企業のデータをどう地図化するか」というプラットフォームの視点を獲得します。教材の `test_parantia_ontology.py` を動かしながら学びます。

🎯 この章の学習ゴール

- オントロジーの3要素(Object / Link / Action)を具体例で説明できる
- OSDK検証スクリプトを実行し、SDKの構造(クライアント・モデル)を読み解ける
- 教材アプリの `data/` を題材に、自分でオントロジーを設計できる

5-1. 概念:オントロジー=業務の地図

Palantir Foundry などのデータ基盤では、バラバラのデータを「業務の言葉」で整理し直します。これがオントロジーです。

要素	意味	data/ フォルダでの例
Object	業務に登場する「モノ」	社員、部署、顧客、サービス、会議(議事録)
Link	モノ同士の「つながり」	社員は部署に 所属する /顧客はサービスを 契約する /社員は会議に 出席する
Action	業務上の「操作」(データを変える行為)	社員を異動させる/契約を更新する

5-2. OSDK検証スクリプトを動かす

1 スクリプトを実行する

教材リポジトリの `test_parantia_ontology.py` は、Foundry SDK(foundry-platform-sdk)のオントロジー関連機能を、**実際のFoundry接続なし**で検証するスクリプトです。

```
pip install foundry-platform-sdk
python test_parantia_ontology.py
```

```
(.venv) PS ...\.company_inner_search_app2> python test_parantia_ontology.py
=== 1. SDK インポート確認 ===
    version = 1.28.0
[PASS] foundry_sdk バージョン確認
[PASS] FoundryClient クラスのインポート
[PASS] OntologiesClient クラスのインポート

=== 2. オントロジーモデル定義確認 ===
[PASS] モデル OntologyObjectV2 が存在する
[PASS] モデル ActionTypeV2 が存在する
...(以下略)
(.venv) PS ...\.company_inner_search_app2> _
```

▲ **画面イメージ:** 検証スクリプトの実行結果。PASS/FAIL形式の出力は第3章で学んだテストの考え方そのもの。「OntologyObjectV2」「ActionTypeV2」など、5-1の概念がSDKのクラス名に対応していることを確認します。

2 スクリプトを読解する

`check(label, fn)` という仕組み(検証の共通化)と、`OntologyClient` / `ObjectTypeClient` / `OntologyObjectClient` の役割分担を読み解き、「SDKはオントロジーをどんな階層で扱っているか」をメモにまとめます。

3 設計演習:data/ のオントロジーを描く

data/ フォルダ(社員・顧客・サービス・議事録)を題材に、Object・Link・Action を洗い出したオントロジー設計図を作成し、レビュー会で発表します。「RAG(文書のまま検索)とオントロジー(構造化して整理)、それぞれ何が得意か」の比較考察を必ず含めます。

✓ 確認ポイント(第5週修了の条件)

- Object / Link / Action を、顧客業務(例:製造業)の例で3つずつ挙げられる
- 「全文検索のRAGだけでは解けない質問」の例(例:集計・追跡系の質問)を挙げ、オントロジーが有効な理由を説明できる

第6章 データパイプライン演習

実案件は「きれいなデータ」から始まらない。data/ 配下の雑多なファイル(PDF / Word / CSV / 議事録)を、検索・分析に使える構造化データへ変換するパイプラインを作ります。

🎯 この章の学習ゴール

- 「収集 → 抽出 → 変換 → 検証 → 出力」というパイプラインの型で処理を設計できる
- 形式の異なるファイルからテキスト・表データを抽出できる
- 「汚いデータ」(表記ゆれ・欠損)への対処を実装できる

6-1. 演習:議事録から「会議マスタ」を作る

1 まずデータの棚卸しをする

コードを書く前に、対象ファイルの一覧と形式・件数・状態(文字化け・命名のばらつき等)を表にします。実案件ではこの棚卸し表が顧客への最初の報告物になります。

```
ls -R data/
```

2 パイプラインを実装する

MTG議事録フォルダの全ファイルを読み、「開催日・会議名・出席者」を抽出して1枚のCSVにまとめます。

pipeline_minutes.py(骨格)

```
import re, csv
from pathlib import Path
from docx import Document          # python-docx: Word文書の読み取り

records, errors = [], []
for path in sorted(Path("data/MTG議事録").glob("*.docx")):
    try:
        text = "\n".join(p.text for p in Document(path).paragraphs)
        record = {
            "ファイル名": path.name,
            # 「2026年6月3日」「2026/06/03」など表記ゆれを吸収して抽出
            "開催日": extract_date(text),
            "会議名": extract_title(text),
            "出席者": extract_members(text),    # 「山田/山田太郎」の名寄せもここで
        }
        validate(record)                    # 欠損チェック(検証は必ず入れる)
        records.append(record)
    except Exception as e:
        errors.append((path.name, str(e)))  # 失敗は握りつぶさず記録する

write_csv("output/meetings.csv", records)
print(f"成功 {len(records)}件 / 失敗 {len(errors)}件")
```

ターミナル — パイプライン実行

```
(.venv) PS ... \company_inner_search_app2> python pipeline_minutes.py
[抽出] 議事録_営業定例_20260603.docx ... OK
[抽出] 議事録_製品開発MTG_0610.docx ... OK
[抽出] mtg_memo_保留.docx ... WARN: 開催日が抽出できず(手動確認リストへ)
成功 11件 / 失敗 1件
output/meetings.csv に出力しました
(.venv) PS ... \company_inner_search_app2> _
```

▲ **画面イメージ:** 実行結果のイメージ。ポイントは「失敗1件」を隠さず出していること。100%自動化に固執せず、機械で9割+人手で1割、と割り切るのも実務の判断です。

品質レポートを添えてPR提出

出力CSVとあわせて「何件処理し、何件失敗し、失敗の原因は何か。データの持ち主に何を確認すべきか」を書いた品質メモを添えます。**発展:**作ったmeetings.csvを教材アプリの検索対象に加え、「先週の営業定例の決定事項は？」が答えられるようになるか試します。

✓ 確認ポイント(第6週修了の条件)

- 全ファイルを一括処理するパイプラインがPRとして提出され、失敗ケースの扱いが実装されている
- 「表記ゆれの名寄せをどこまで自動化すべきか」について自分の判断基準を言える

第7章 クラウドデプロイ基礎

「自分のPCでしか動かない」を卒業する週。教材アプリを Streamlit Community Cloud にデプロイし、環境変数・シークレット管理を実践します。

🎯 この章の学習ゴール

- ローカルとクラウドの環境差(パス・環境変数・リソース)を意識した構成にできる
- APIキーをシークレット機能で安全に設定できる(コードに書かない)
- デプロイ後の動作確認・ログ確認・更新反映の流れを一人で回せる

7-1. デプロイ手順

1 デプロイ前チェック

- `requirements` ファイルが最新か(ローカルで `pip install -r` し直して起動確認)
- `.env` や APIキーがリポジトリに含まれていないか(`git log --all -- .env` で履歴も確認)
- デプロイ用ブランチを最新のmainから作成

2 Streamlit Community Cloud でアプリを作成する

share.streamlit.io にGitHubアカウントでログイン → 「New app」 → リポジトリ・ブランチ・`main.py` を指定します。

3 シークレットを設定してデプロイする

「Advanced settings → Secrets」に APIキーを設定します。ローカルの `.env` に相当するものをクラウド側の金庫に入れるイメージです。

Streamlit Community Cloud — Deploy an app

← → ↺ https://share.streamlit.io/deploy

Deploy an app

Repository

koheishikari/company_inner_search_app2

Branch / Main file path

deploy/training / main.py

Advanced settings → Secrets

OPENAI_API_KEY = "sk-*****"

Deploy!

🔔 Your app is in the oven... 数分でビルドが完了し、公開URL(<https://...streamlit.app>)が発行されます

▲ **画面イメージ:** デプロイ設定画面のイメージ。Secrets欄に入れた値はアプリから環境変数として読めます。公開URLは研修内限定で共有し、動作確認後は不要なら停止します。

4 デプロイ後の確認と更新

- 公開URLで質問を3種類実行し、ローカルと同じ回答が返るか確認
- 管理画面のログ(Manage app)でエラーが出ていないか確認
- 小さな修正をpushし、**自動で再デプロイされることを確認**(=継続的デリバリーの入口)

🔧 つまづきポイントと対処

症状	原因と対処
ローカルでは動くのにクラウドで起動失敗	requirements の記載漏れが大半。ビルドログの <code>ModuleNotFoundError</code> を確認して追記
<code>AuthenticationError</code> が出る	Secretsのキー名がコード側の環境変数名と一致していない。つづりを確認
メモリ不足で落ちる	無料枠はリソースが小さい。data/ を研修用の小さいサブセットに差し替える

第8章 コンサルティング実践

教科書どおりに進まない実戦形式。「要件が曖昧で、途中で変わる」ロールプレイと、先輩FDEの現場を観察するシャドーイングの2本立てです。

🎯 この章の学習ゴール

- 要件変更の要求に対し、感情的にならずスコープ交渉で着地できる
- 「技術的にできること」と「顧客が本当に必要なこと」のギャップを提案書で埋められる
- 先輩FDEの立ち居振る舞いから、質問設計・宿題の引き取り方を盗む

8-1. 実戦型ロールプレイ(要件が変わる設定)

顧客役(先輩FDE)との面談を3回実施します。2回目の面談で必ず要件変更が飛んでくるシナリオです。

回	顧客役の動き(受講者には事前非公開)	受講者に求められる対応
1回目	「とにかく資料探しが大変で…」と曖昧に困りごとを語る	ヒアリングで課題を特定し、要件定義書を作成
2回目	「上司がスマホ対応も必須と言っている。あと来週までにできない？」と要求追加	影響(工数・品質)を説明し、優先順位とスコープを再交渉
3回目	再交渉の結果に対し、納得 or さらに揺さぶり	合意事項を文書化し、その場で読み合わせて確定

スコープ交渉の基本フレーズ(演習で実際に口に出して練習)

言ってはいけない	言い換える
「それは無理です」	「実現には○日かかり、当初予定の△△が遅れます。どちらを優先しますか？」(選択肢を返す)
「たぶんできると思います」(安請け合い)	「持ち帰って影響を確認し、明日までに可否と条件をご連絡します」(宿題として引き取る)
「前回そう決まりましたよね」(正論で押す)	「状況が変わったのですね。目的に立ち返ると〜が最優先という理解で合っていますか？」(目的に戻す)

8-2. シャドーイング(週2回以上)

1 観察ポイントを決めてから同席する

先輩FDEの商談・定例に同席します。漫然と聞かず、毎回1つ観察テーマを決めます(例:「難しい質問をどうかわすか」「会議の最後5分で何をするか」)。

2 気づきメモを当日中に提出し、メンターと振り返る

「観察したこと(事実)/なぜそうしたと思うか(解釈)/自分ならどうするか(適用)」の3段構成で書きます。

✓ 確認ポイント(第8～9週修了の条件)

- ロールプレイ3回を完走し、最終的な合意文書に顧客役のサインを得た
- 要件変更時に、その場で安請け合いも拒絶もしなかった(録画で振り返り)
- シャドーイング気づきメモが4件以上提出されている

第9章 模擬プロジェクト

3～4名のチームで実在の社内課題に取り組む総仕上げ。週次で顧客役(マネージャー)にデモし、フィードバックを反映しながら完成に向かいます。

🎯 プロジェクト概要

- **題材例:**「営業部の過去提案書検索の効率化」(実在の社内課題から選定)
- **体制:**3～4名チーム。リーダー・顧客窓口・実装リードなどの役割は週替わりで交代
- **ルール:**全員が最低1回は顧客役の前で話す/成果はすべてIssue・PRに記録する

9-1. 3週間の進め方

週	活動	マイルストーン(毎週金曜)
第10週	ヒアリング → 要件定義 → タスク分解	キックオフデモ:要件定義書の承認とタスクボードの公開
第11週	実装スプリント①	中間デモ:コア機能が動く状態で顧客役のフィードバックを得る
第12週	実装スプリント② → リハーサル	最終発表会:デモ20分+質疑15分/個人振り返り+360度フィードバック

9-2. チーム運営の道具

1 タスクボードで「誰が何をしているか」を見える化する

GitHub Projects(かんばん方式)でタスクを管理します。**1タスク=1 Issue=完了までおおむね1日以内**のサイズに分解するのがコツ。大きすぎるタスクは「進捗90%」のまま停滞します。



▲ **画面イメージ:** タスクボードの画面イメージ。毎朝15分の朝会をこのボードの前で行い、「昨日やったこと・今日やること・困っていること」を1人1分で共有します。

2 週次デモは「顧客の業務シナリオ」で行う

機能一覧の紹介ではなく、「営業の田中さんが明日の商談前に類似提案を探す」という**業務の物語**に沿ってデモします。フィードバックはその場でIssue化し、翌週の計画に反映します。

3 最終発表と360度フィードバック

最終週は成果発表会(デモ20分+質疑15分)。終了後、チームメンバー・メンター・顧客役の全員から「続けるべき点/変えるべき点」のフィードバックを受け、個人の振り返りシートにまとめます。これが配属面談の資料になります。

✓ 修了判定(カリキュラム共通基準)

- 顧客役へのヒアリングから要件定義書を作成できる
- 教材アプリ相当のRAGアプリを自力で改修・機能追加できる
- 成果物を顧客役の前でデモ・プレゼンし、質疑に答えられる

修了者は配属後3ヶ月間、月1回のフォローアップ会に参加します。模擬プロジェクトのIssue・PR・発表資料はポートフォリオとして蓄積され、アサイン判断に活用されます。